

Resumen de la Unidad 3

Esta unidad introduce el uso de herramientas de control de versiones, específicamente **Git** y **GitHub**, los cuales permiten gestionar cambios en proyectos de software, trabajar en equipo y mantener un historial de modificaciones.

Introducción

El control de versiones es una práctica esencial en el desarrollo de software. Permite registrar, organizar y recuperar cambios realizados en un proyecto a lo largo del tiempo. Sin un sistema de control de versiones, los desarrolladores tendrían que gestionar manualmente múltiples copias de su código, lo que puede generar confusión y errores.

Para resolver este problema, se utilizan herramientas como **Git**, un sistema de control de versiones distribuido, y **GitHub**, una plataforma basada en la nube que permite alojar repositorios y colaborar en proyectos de manera eficiente.

Video: Controlador de Versiones

En el video de apoyo se explica el concepto de control de versiones, su importancia en la programación y cómo herramientas como Git facilitan la gestión de proyectos, especialmente cuando varias personas trabajan en un mismo código.

GIT

Git es un sistema de control de versiones distribuido que permite rastrear cambios en archivos y coordinar el trabajo entre múltiples desarrolladores.

Características principales de Git:

- Permite registrar todos los cambios realizados en un proyecto.
- Facilita la colaboración entre varios programadores sin que se sobrescriban los cambios de otros.
- Funciona de manera distribuida, lo que significa que cada desarrollador tiene una copia completa del historial del proyecto.
- Permite trabajar en diferentes versiones del código (ramas) y combinarlas cuando sea necesario.

GIT (2)

Git trabaja con tres áreas principales:

1. **Working Directory (Directorio de trabajo):** Es donde se encuentran los archivos en el sistema de archivos del usuario.
2. **Staging Area (Área de preparación):** Es donde se colocan los archivos antes de confirmarlos en el historial.
3. **Repository (Repositorio):** Es donde Git guarda todos los cambios confirmados en el historial del proyecto.

Los comandos básicos de Git incluyen:

- `git init` → Inicializa un nuevo repositorio.
- `git clone <URL>` → Clona un repositorio existente desde GitHub.
- `git status` → Muestra el estado actual de los archivos.
- `git add .` → Agrega los cambios al área de preparación.

- `git commit -m "Mensaje"` → Guarda los cambios en el historial del repositorio.
- `git push origin <rama>` → Envía los cambios a GitHub.
- `git pull origin <rama>` → Descarga los últimos cambios desde GitHub.

GITHUB

GitHub es una plataforma que permite alojar repositorios Git en la nube, facilitando la colaboración entre desarrolladores.

Diferencias entre Git y GitHub:

- **Git** es un sistema de control de versiones que se ejecuta en la computadora del usuario.
- **GitHub** es un servicio en línea que permite almacenar repositorios Git y trabajar en equipo.

Funciones principales de GitHub:

- Permite compartir código con otros programadores.
- Facilita la gestión de proyectos a través de repositorios públicos y privados.
- Ofrece herramientas como *Issues* y *Pull Requests* para administrar cambios y reportar problemas en el código.
- Permite la integración con herramientas de automatización y despliegue continuo.

Instrucciones básicas de Git

Para trabajar con Git y GitHub, es fundamental seguir un flujo de trabajo ordenado:

1. Crear un nuevo repositorio en GitHub.
2. Clonar el repositorio en la computadora local con `git clone <URL>`.
3. Hacer cambios en los archivos y verificar el estado con `git status`.
4. Agregar los archivos al área de preparación con `git add ..`
5. Guardar los cambios en el historial con `git commit -m "Descripción del cambio"`.
6. Subir los cambios a GitHub con `git push origin <rama>`.

Ejemplos de uso de GitHub

Los ejemplos prácticos de GitHub incluyen:

- **Creación de un repositorio y subida de archivos.**
- **Manejo de ramas:** Permite trabajar en diferentes versiones del código sin afectar la versión principal.
- **Colaboración en proyectos:** GitHub permite que varios desarrolladores trabajen en el mismo proyecto, realicen cambios y fusionen sus contribuciones mediante *Pull Requests*.
- **Uso de Issues y Wikis:** GitHub proporciona herramientas para reportar problemas y documentar proyectos.

Conclusión

La tercera unidad introduce el uso de Git y GitHub, herramientas esenciales para el desarrollo de software. Git permite gestionar versiones de código de manera local, mientras que GitHub facilita la colaboración en proyectos alojados en la nube. Comprender el flujo de trabajo en Git y dominar los comandos básicos son habilidades clave para cualquier programador que desee trabajar de manera eficiente en equipo y mantener un control organizado de su código.